



Spring – Difference Between BeanFactory and ApplicationContext

Y yaminitalisetty

Read

Discuss

Courses

Practice

Video

Spring is one of the most popular Java EE frameworks. It is an open-source lightweight framework that allows Java EE developers to build simple, reliable, and scalable enterprise applications. It provides Aspect-oriented programming. It provides support for all generic and middleware services and mainly focuses on providing various ways to help you manage your business objects. It is a modularized framework where all the modules are designed on the concept called “Dependency Injection”.

Dependency Injection: Dependency Injection is a design pattern that allows the spring container to ‘inject’ objects into other objects or dependencies. In simple words, the control of creating objects and managing the spring components is taken care of by the Spring containers.

Spring Containers

Spring Framework provides two of the most fundamental and important packages, they are the **org.springframework.beans** and **org.springframework.context** packages. Code in these packages provides the basis for Spring’s **Inversion of Control/Dependency Injection** features. Spring containers are responsible for creating bean objects and injecting them into the classes. The two containers are namely,

1. **BeanFactory(I)** – Available in org.springframework.bea  ctory package.

2. **ApplicationContext(I)** – Available in `org.springframework.context` package.

The BeanFactory Interface

This is the root interface for accessing a Spring bean container. It is the actual container that instantiates, configures, and manages a number of beans. These beans collaborate with one another and thus have dependencies between themselves. These dependencies are reflected in the configuration data used by the BeanFactory. This interface is implemented by the objects that hold a number of bean definitions, each uniquely identified by a String name. The most common implementation class used for this BeanFactory is **XmlBeanFactory** available in `org.springframework.beans.factory.xml` package.

***Note:** BeanFactory is deprecated from Spring 3.0.*

Example code:

Java



```
ClassPathResource resource = new ClassPathResource("beans.xml");  
XmlBeanFactory factory = new XmlBeanFactory(resource);
```

The ApplicationContext Interface

This interface is designed on top of the BeanFactory interface. The ApplicationContext interface is the advanced container that enhances BeanFactory functionality in a more framework-oriented style. While the BeanFactory provides basic functionality for managing and manipulating beans, often in a programmatic way, the ApplicationContext provides extra functionality like MessageSource, Access to resources, Event propagation to beans, Loading of multiple (hierarchical) contexts etc. There are so many implementation classes that can be used such as **ClassPathXmlApplicationContext**, **FileSystemXmlApplicationContext**, **AnnotationConfigWebApplicationContext** etc.

Example code:

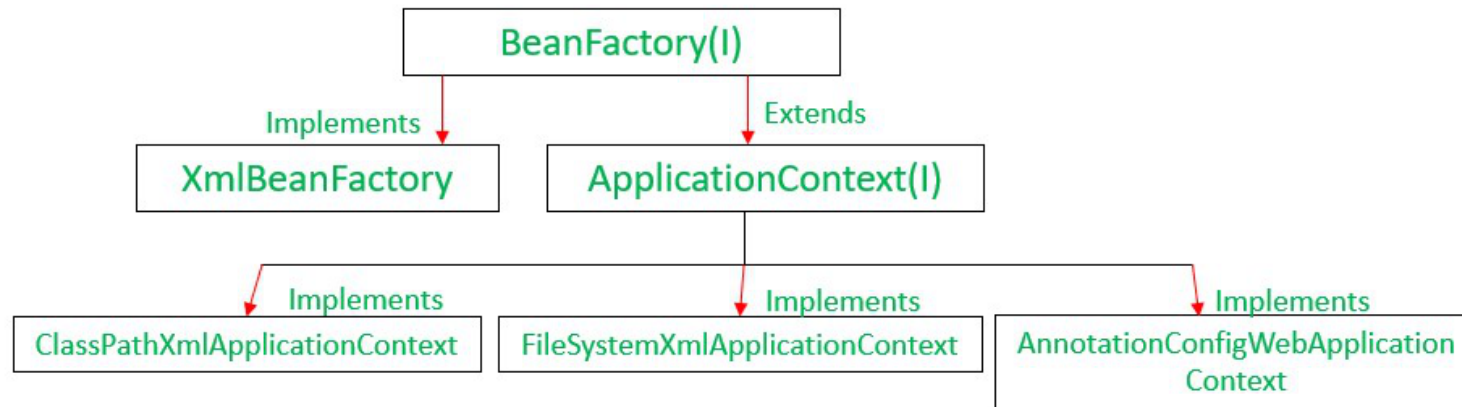
Java



```
ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
```

Interface Hierarchy

Below is the hierarchy of the BeanFactory(I) and ApplicationContext(I) with some of their implementation classes.



Hierarchy Structure

Difference Table

BeanFactory	ApplicationContext
It is a fundamental container that provides the basic functionality for managing beans.	It is an advanced container that extends the BeanFactory that provides all basic functionality and adds some advanced features.
It is suitable to build standalone applications.	It is suitable to build Web applications, integration with AOP modules, ORM and distributed applications.

[Java Arrays](#)
[Java Strings](#)
[Java OOPs](#)
[Java Collection](#)
[Java 8 Tutorial](#)
[Java Multithreading](#)
[Java Exception Handling](#)
[Java Programs](#)
[Java Project](#)
[Java](#)

BeanFactory	ApplicationContext
It supports only Singleton and Prototype bean scopes.	It supports all types of bean scopes such as Singleton, Prototype, Request, Session etc.
It does not support Annotations. In Bean Autowiring, we need to configure the properties in XML file only.	It supports Annotation based configuration in Bean Autowiring.
This interface does not provides messaging (i18n or internationalization) functionality.	ApplicationContext interface extends MessageSource interface, thus it provides messaging (i18n or internationalization) functionality.
BeanFactory does not support Event publication functionality.	Event handling in the ApplicationContext is provided through the ApplicationEvent class and ApplicationListener interface.
In BeanFactory, we need to manually register BeanPostProcessors and BeanFactoryPostProcessors.	The ApplicationContext automatically registers BeanFactoryPostProcessor and BeanPostProcessor at startup.
BeanFactory will create a bean object when the getBean() method is called thus making it Lazy initialization.	ApplicationContext loads all the beans and creates objects at the time of startup only thus making it Eager initialization.

BeanFactory	ApplicationContext
BeanFactory interface provides basic features only thus requires less memory. For standalone applications where the basic features are enough and when memory consumption is critical, we can use BeanFactory.	ApplicationContext provides all the basic features and advanced features, including several that are geared towards enterprise applications thus requires more memory.

Explanation of Functionalities

1) Bean Autowiring

Spring container is able to autowire relationships between collaborating beans. It means that it is possible to automatically let Spring resolve other beans for your bean by inspecting the contents of the Container. Autowiring is specified per bean and thus can be enabled for some beans, while other beans will not be autowired. Using autowiring, it is possible to reduce or eliminate the need to specify properties or constructor arguments.

Autowiring in BeanFactory:

In BeanFactory, the autowire mode for a bean definition is specified by using the '**autowire**' attribute of the **<bean/>** element in an XML-based configuration file.

Example:

XML



```
<bean id="welcomeBean" class="com.geeks.beans.WelcomeBean" autowire="byName"/>
```

Autowiring in ApplicationContext:

In ApplicationContext, we can use the annotation “**@Autowired**” on top of properties or setter methods in the bean class.

Example:

Java



```
@Autowired  
private String name;
```

2) Registering BeanPostProcessors and BeanFactoryPostProcessors

BeanPostProcessors: A Bean post-processor is a java class that implements the `org.springframework.beans.factory.config.BeanPostProcessor` interface, which consists of two callback methods. It allows custom modification of new bean instances created by spring containers. If we want to implement some custom logic before or after the Spring container finishes instantiating, configuring, and initializing a bean, we can plug in one or more `BeanPostProcessor` implementations.

BeanFactoryPostProcessors: A Bean factory post-processor is a java class which implements the `org.springframework.beans.factory.config.BeanFactoryPostProcessor` interface. It works on the bean definitions or configuration meta data of the bean before beans are actually created. These are invoked to resolve run time dependencies.

Registering in BeanFactory:

While using the `BeanFactory` interface, if there are any beans that implement the `BeanPostProcessor` or `BeanFactoryPostProcessor` interfaces, bean post-processors have to manually be explicitly registered.

Example:

Java

```
// create BeanFactory
ConfigurableBeanFactory factory = new ConfigurableBeanFactory(resource);

// register needed BeanPostProcessors
HelloBeanPostProcessor bpp = new HelloBeanPostProcessor();
factory.addBeanPostProcessor(bpp);
```

Registering in ApplicationContext:

An ApplicationContext interface will automatically detect if any beans that are deployed into it implementing the BeanPostProcessor or BeanFactoryPostProcessor interfaces and register them as post-processors, and then they are called appropriately by the factory on bean creation. So, it is much more convenient to use bean factory post-processors in ApplicationContexts than in plain BeanFactories.

3) Loading/Initializing the beans

We will see a simple example to understand the difference between BeanFactory and ApplicationContext containers.

BeanFactory Example:

Create below classes and XML files.

WelcomeBean.java: A Java bean class to define the properties, getter/setter methods of the properties.

Java

```
public class WelcomeBean {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String welcomeMsg() {  
        return "Welcome " + name;  
    }  
  
    public void initializeBean() {  
        System.out.println("Welcome Bean is initialized!!");  
    }  
  
    public void destroyBean() {  
        System.out.println("Welcome Bean is destroyed!!");  
    }  
}
```

```
}
```

beans.xml: XML file to configure the beans.

XML

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="welcomeBean" class="WelcomeBean"
          init-method="initializeBean" destroy-method="destroyBean">
        <property name="name" value="geek" />
    </bean>

</beans>
```

WelcomeBeanTest.java: Main class to create the BeanFactory object and run the application.

Java

```
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;

import com.geeks.beans.WelcomeBean;

public class WelcomeBeanTest {
    public static void main(String[] args) {

        ClassPathResource res = new ClassPathResource("beans.xml");
        XmlBeanFactory factory = new XmlBeanFactory(res);
        System.out.println("Before getBean() method");

        WelcomeBean welcome = (WelcomeBean) factory.getBean("welcomeBean");
        System.out.println(welcome.welcomeMsg());
        ((XmlBeanFactory)factory).destroySingletons();

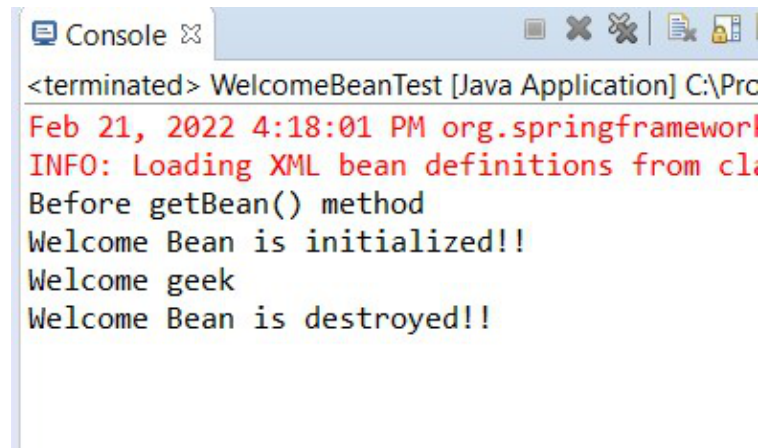
    }
}
```

Execution:

- In this example, we are calling **welcomeMsg()** method to print the message in the console.
- In bean class, we specified the methods, **initializeBean()** as **init-method** and **destroyBean()** as **destroy-method**. This configuration is specified in **beans.xml** file.
- When we run the application, the main method will be executed. In that, first, it will create the resource object to load the XML file, then XmlBeanFactory object is created.
- And then the application prints, **Before getBean() method** in the console. Now, the getBean() method is called, at this point, the container will load the bean and initialize the bean class. Until the execution comes at getBean()

method, the BeanFactory container will not initialize the bean. Hence, it is called **Lazy Initialization**.

- So, the output will be as follows,



```
Console
<terminated> WelcomeBeanTest [Java Application] C:\Pro
Feb 21, 2022 4:18:01 PM org.springframework
INFO: Loading XML bean definitions from cla
Before getBean() method
Welcome Bean is initialized!!
Welcome geek
Welcome Bean is destroyed!!
```

Output

ApplicationContext Example:

Now, we will create the same application using ApplicationContext.

HelloBean.java: A Java bean class to define the properties, getter/setter methods of the properties.

Java

```
public class HelloBean {
```



```
private String name;

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String helloMsg() {
    return "Hello " + name;
}

public void initializeBean() {
    System.out.println("Hello Bean is initialized!!");
}

public void destroyBean() {
    System.out.println("Hello Bean is destroyed!!");
}
}
```

applicationContext.xml: XML file to configure the beans.

Java



```
public class WelcomeBean {
```



```
    private String name;
```

```
public String getName() {  
    return name;  
}  
  
public void setName(String name) {  
    this.name = name;  
}  
  
public String welcomeMsg() {  
    return "Welcome " + name;  
}  
  
public void initializeBean() {  
    System.out.println("Welcome Bean is initialized!!");  
}  
  
public void destroyBean() {  
    System.out.println("Welcome Bean is destroyed!!");  
}  
}
```

HelloBeanTest.java: Main class to create the ApplicationContext object and run the application.

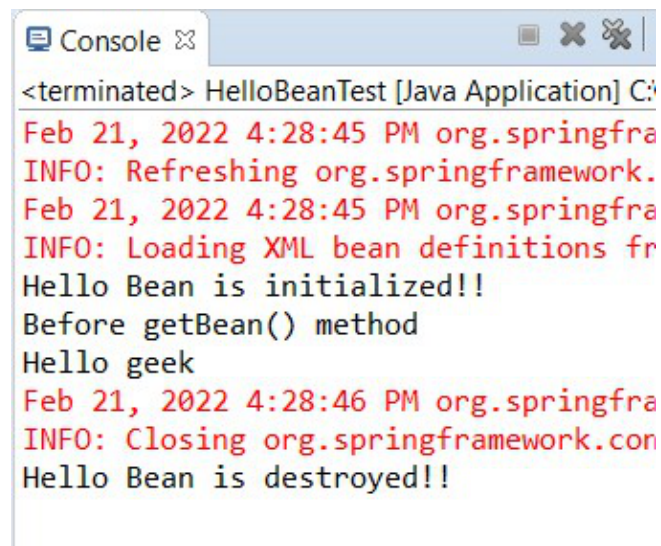
Java

```
 import org.springframework.context.ApplicationContext;  
 import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
import com.geeks.beans.HelloBean;
```

```
public class HelloBeanTest {  
  
    public static void main(String[] args) {  
  
        ApplicationContext con = new ClassPathXmlApplicationContext("applicationContext.xml");  
        System.out.println("Before getBean() method");  
        HelloBean hello = (HelloBean) con.getBean("helloBean");  
  
        System.out.println(hello.helloMsg());  
        ((ClassPathXmlApplicationContext) con).close();  
  
    }  
  
}
```

Execution:

- In this example, we are calling **helloMsg()** method to print the message in the console.
- In bean class, we specified the methods, **initializeBean()** as init-method and **destroyBean()** as destroy-method. This configuration is specified in **applicationContext.xml** file.
- When we run the application, the main method will be executed. In that, first, it will the ApplicationContext object and now itself the container will identify all the bean objects and it will initialize before calling getBean() method. Hence, it is called **Eager Initialization**.
- It prints the initialization message first before calling the getBean() method.
- So, the output will be as follows,



```
<terminated> HelloBeanTest [Java Application] C:\
Feb 21, 2022 4:28:45 PM org.springframework
INFO: Refreshing org.springframework.
Feb 21, 2022 4:28:45 PM org.springframework
INFO: Loading XML bean definitions fr
Hello Bean is initialized!!
Before getBean() method
Hello geek
Feb 21, 2022 4:28:46 PM org.springframework
INFO: Closing org.springframework.con
Hello Bean is destroyed!!
```

Output

If you compare both the outputs, we can see that using BeanFactory, the beans will be initialized when the `getBean()` method is called, which means on-demand. But the ApplicationContext will initialize all the beans at the startup only.

4) Internationalization Functionality in ApplicationContext

The ApplicationContext interface extends the interfaces called **MessageSource** and **NestingMessageSource**, and therefore provides messaging (i18n) or internationalization functionality. It is capable of resolving hierarchical messages.

Example:

Java

```
String getMessage (String code, Object[] args, String default, Locale loc)
```

This is the basic method used to retrieve a message from the MessageSource. When no message is found for the specified locale, the default message is used. When the ApplicationContext gets loaded, it automatically searches for a MessageSource bean that is defined in the context. The bean should have the name **messageSource**. Currently, Spring provides two MessageSource implementations. They are the **ResourceBundleMessageSource** and the **StaticMessageSource**.

5) Events Propagation in ApplicationContext

ApplicationContext provides Event handling through the **ApplicationEvent** class and **ApplicationListener** interface. If a bean implements the ApplicationListener interface and is deployed into the context, every time an ApplicationEvent gets published to the ApplicationContext, that bean will be notified. This is the standard **Observer** design pattern.

Spring provides three standard events namely ContextRefreshedEvent, ContextClosedEvent and RequestHandledEvent. We can also create custom events by calling the **publishEvent()** method on the ApplicationContext and specifying a parameter which is an instance of our custom event class implementing ApplicationEvent.

Conclusion

As we discussed above, both ApplicationContext and BeanFactory containers are used to create and manage the bean objects. BeanFactory is the fundamental interface that provides all the basic functionality to create and manage the bean objects and the ApplicationContext interface extends the BeanFactory interface, it provides all the basic functionality and also some advanced features like the ability to load file resources in a generic fashion, to publish events to registered listeners, to resolve messages, supporting internationalization, etc.

Last Updated : 28 Nov, 2022

8

Similar Reads



Difference Between ApplicationContext and WebApplicationContext in Spring MVC



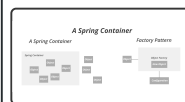
Spring - ApplicationContext



Spring Boot - Spring JDBC vs Spring Data JDBC



Difference between Spring MVC and Spring Boot



Spring - BeanFactory



Difference Between Spring DAO vs Spring ORM vs Spring JDBC



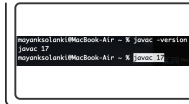
Difference between Spring and Spring Boot



Difference Between Spring Boot Starter Web and Spring Boot Starter Tomcat



How to Create a Spring Boot Project in Spring Initializr and Run it in IntelliJ IDEA?



How to Download and Install Spring Tool Suite (Spring Tools 4 for Eclipse) IDE?

Related Tutorials



Spring MVC Tutorial



Spring Tutorial



Spring Boot Tutorial



Java 8 Features - Complete Tutorial



Introduction to Heap - Data Structure and Algorithm Tutorials

Previous

Spring - ApplicationContext

Next

Spring Dependency Injection with Example

Article Contributed By :

Y

yaminitalisetty

yaminitalisetty

Follow

Vote for difficulty

Current difficulty : Basic

Easy

Normal

Medium

Hard

Expert

Improved By : [kk773572498](#), [germanshepherd48](#), [ahaodeyop4yr](#)

Article Tags : [Java-Spring](#), [Picked](#), [Difference Between](#), [Java](#)

Practice Tags : [Java](#)

[Improve Article](#)[Report Issue](#)

A-143, 9th Floor, Sovereign Corporate
Tower, Sector-136, Noida, Uttar Pradesh -
201305

feedback@geeksforgeeks.org



Company

[About Us](#)[Legal](#)[Careers](#)[In Media](#)

Explore

[Job-A-Thon For Freshers](#)[Job-A-Thon For](#)[Experienced](#)[GfG Weekly Contest](#)

Languages

[Python](#)[Java](#)[C++](#)[PHP](#)

DSA Concepts

[Data Structures](#)[Arrays](#)[Strings](#)[Linked List](#)

DSA Roadmaps

[DSA for Beginners](#)[Basic DSA Coding](#)[Problems](#)

Web Development

[HTML](#)[CSS](#)[JavaScript](#)[Bootstrap](#)

[Contact Us](#)[Advertise with us](#)[Offline Classes](#)[\(Delhi/NCR\)](#)[DSA in JAVA/C++](#)[Master System Design](#)[Master CP](#)[GoLang](#)[SQL](#)[R Language](#)[Android Tutorial](#)[Algorithms](#)[Searching](#)[Sorting](#)[Mathematical](#)[Dynamic Programming](#)[Complete Roadmap To](#)[Learn DSA](#)[DSA for FrontEnd](#)[Developers](#)[DSA with JavaScript](#)[Top 100 DSA Interview Problems](#)[ReactJS](#)[AngularJS](#)[NodeJS](#)

Computer Science

[GATE CS Notes](#)[Operating Systems](#)[Computer Network](#)[Database Management System](#)[Software Engineering](#)[Digital Logic Design](#)[Engineering Maths](#)

Python

[Python Programming Examples](#)[Django Tutorial](#)[Python Projects](#)[Python Tkinter](#)[OpenCV Python Tutorial](#)[Python Interview](#)[Question](#)

Data Science & ML

[Data Science With Python](#)[Data Science For Beginner](#)[Machine Learning Tutorial](#)[Maths For Machine Learning](#)[Pandas Tutorial](#)[NumPy Tutorial](#)[NLP Tutorial](#)[Deep Learning Tutorial](#)

DevOps

[Git](#)[AWS](#)[Docker](#)[Kubernetes](#)[Azure](#)[GCP](#)

Competitive Programming

[Top DSA for CP](#)[Top 50 Tree Problems](#)[Top 50 Graph Problems](#)[Top 50 Array Problems](#)[Top 50 String Problems](#)[Top 50 DP Problems](#)[Top 15 Websites for CP](#)

System Design

[What is System Design](#)[Monolithic and](#)[Distributed SD](#)[Scalability in SD](#)[Databases in SD](#)[High Level Design or HLD](#)[Low Level Design or LLD](#)[Top SD Interview Questions](#)

Interview Corner

[Company Wise Preparation](#)[Preparation for SDE](#)

GfG School

[CBSE Notes for Class 8](#)[CBSE Notes for Class 9](#)[CBSE Notes for Class 10](#)

Commerce

[Accountancy](#)[Business Studies](#)[Economics](#)

UPSC

[Polity Notes](#)[Geography Notes](#)[History Notes](#)

SSC/ BANKING

[SSC CGL Syllabus](#)[SBI PO Syllabus](#)[SBI Clerk Syllabus](#)

Write & Earn

[Write an Article](#)[Improve an Article](#)[Pick Topics to Write](#)

Experienced Interviews	CBSE Notes for Class 11	Management	Science and Technology	IBPS PO Syllabus	Write Interview
Internship Interviews	CBSE Notes for Class 12	Income Tax	Notes	IBPS Clerk Syllabus	Experience
Competitive	English Grammar	Finance	Economics Notes	Aptitude Questions	Internships
Programming			Important Topics in Ethics	SSC CGL Practice Papers	
Aptitude Preparation			UPSC Previous Year Papers		

@geeksforgeeks , Some rights reserved